Code Review — Junior's Webhook Handler

Reviewer: Senior Backend Engineer · **Severity scale:** P0 Critical → P3 Low **Subject:** `webhook(req)` — production webhook endpoint for gateway payment events

> Reviewed cold, as it would ship: handler mounted at `POST /webhook`, writing directly to the production order database and emailing customers inline. Every finding below is a real defect or risk in the snippet as written. Ranked by severity; within a tier, most dangerous first.

P0 — Critical: money is wrong or can be forged

1. Floating-point money arithmetic — `float(e["amount"])`, `order.paid`, `order.refunded += …`

`float` cannot represent most decimal rupee values exactly (`0.1` is `0.1000000000000000055511151231257827021181583404541015625` in binary). Sums of floats drift by paise; equality tests (`order.refunded == order.paid`) then pass or fail at random. Over 50M events, paise-level drift compounds into real settlement discrepancies. **Fix:** integer paise end-to-end (or `Decimal`); format to ₹ only at the presentation edge.

2. No idempotency — at-least-once delivery is treated as exactly-once

Gateway webhooks are delivered **at least once**. The handler happily applies `payment.captured` or `refund.processed` again on every retry: orders get double-charged in the books, refunds double-counted, emails re-sent. There is no `processed_events` table, no unique constraint on `event_id`, no dedupe check of any kind. This is the single most common way payment systems lose track of money. **Fix:** unique index on `event_id`; insert-first in the same transaction as the state change; duplicates are acknowledged with `200` and ignored.

3. Events are applied in arrival order, not event time

`if e["sent_at"] > order.last_event_at` only decorates the `last_event_at` field — it never reorders the application of events. A capture delivered before its authorization still flips `status = "CAPTURED"`; worse, a `refund.processed` arriving (in the log) before an older `payment.captured` gets applied, and is then silently discarded by the unconditional `order.status = "CAPTURED"` overwrite below. Final states are then wrong and non-deterministic across replays. **Fix:** process events per order in `sent_at` order — either from an ordered event store at read time, or by recomputing order state from the full logged event set on each delivery.

4. `api_version` is never read — v1 and v2 amounts are mixed as floats

Gateway contract: **v1 `amount` is rupees (decimal string), v2 is paise (integer)**. This handler parses both with `float()` — so a v2 amount of `249900` (i.e. ₹2,499.00) is recorded as ₹249,900.00, a **100× overstatement**, and a v1 `"2499.00"` string coerces to 2499. Mixed-version traffic corrupts every downstream total. **Fix:** branch on `api_version`, normalize to integer paise at the edge, reject unknown versions.

5. `refund.created` moves money — it must not

The gateway contract says a refund moves money only when `refund.processed` arrives; `refund.created` is **pending**. This handler adds the pending amount to `order.refunded` immediately — inflating refunds, understating settlement, and potentially marking orders `REFUNDED` that were paid. **Fix:** count `refund.created` as a pending-refund marker only; touch money only on `refund.processed`.

6. No webhook authenticity check — anyone can forge `payment.captured`

The handler accepts any POST with a JSON body. An attacker (or a misconfigured partner) can mark orders paid. **Fix:** verify the gateway's HMAC-SHA256 signature over the raw body using a shared secret, with a constant-time comparison, before parsing; reject with `401` on mismatch.

P1 — High: integrity, races, and missing rules

7. `payment.captured` unconditionally overwrites state

`order.status = "CAPTURED"` executes even if a refund was already processed (out-of-order delivery), clobbering `REFUNDED` / `PARTIALLY_REFUNDED`. The handler must recompute terminal state from the full event history, or at minimum never downgrade a refund state.

8. Lost update under concurrency — no transaction, no lock

Read-modify-write on `order` (`db.get_order` → mutate → `db.save`) with no row lock (`SELECT … FOR UPDATE`) or optimistic version check. Two events for the same order on two workers interleave: both read the same `order.refunded`, both add, one overwrite wins — one refund silently vanishes. **Fix:** single transaction; lock the order row (or an advisory lock on `order_id`); or make updates commutative (additive columns) with a version guard.

9. Failure-after-capture rule is not implemented — `payment.failed` has no branch at all

The brief's state rules hinge on it: a failure after capture is **stale and must be ignored**; a failure before a later authorization/capture is overridden. This handler ignores `payment.failed` entirely (failures are invisible), and has no concept of stale events.

10. Orphan events crash the handler — and 500s trigger infinite delivery storms

`db.get_order(e["order_id"])` returning `None` raises `AttributeError` on the next line → unhandled 500 → gateway retries forever (poison message). The brief explicitly requires orphans to be reported, not processed. **Fix:** aggregate: if the order is unknown, persist to an `orphan_events` report, acknowledge `200`, stop.

11. Full-refund detection by float equality — and no `>=`

`order.refunded == order.paid` with floats is a coin flip near equality, and misses over-refunds. Brief requires "refunded below / at least the captured amount" → `>=` on integer paise.

12. Customer email sent inline, before commit, unconditionally

Sending mail in the request path couples settlement to the mail provider (latency, timeouts, outages); doing it **before** `db.save` risks emailing a "CAPTURED" confirmation for an order whose write then failed; and every duplicate delivery re-sends it — no dedupe key. **Fix:** enqueue (outbox pattern) after a successful commit; workers send asynchronously with idempotency on `(order_id, status, channel)`.

13. `last_event_at` updated even for stale/duplicate events, compared as strings

String comparison `"2026-06-01T00:00:00+05:30" > "2026-06-01T00:00:00Z"` is format-dependent and wrong for equal instants in different offsets; mixing naive and aware timestamps raises `TypeError`. Compare parsed instants (epoch micros / `timestamptz`), and don't advance watermarks on ignored duplicates.

P2 — Medium: operability & robustness

14. No input validation or size guard

Missing keys → `KeyError` → 500 → gateway retries (see #10); negative `amount` accepted; unknown `type` silently ignored (status never updated but email still sent saying something changed); no body size limit. **Fix:** strict schema validation (400 on defect), allow-listed types/versions, max body size.

15. Always `return 200`

Including for malformed payloads and unauthenticated requests — the gateway can't distinguish "accepted" from "garbage accepted"; genuinely poisoned messages are never dead-lettered. Return `202/200` for accepted (including duplicates), `400` malformed, `401` bad signature, `503` transient failure the gateway should retry.

16. EXPIRED state missing entirely

"Authorized but never captured, older than 7 days at cut-off" → `EXPIRED` is unimplemented; authorized-then-lapsed orders stay `AUTHORIZED` forever, inflating the pipeline.

17. Captured-vs-order amount mismatch never flagged

Brief requires the mismatch flag (order keeps state, gets flagged). Absent entirely — Finance loses the reconciliation signal the whole exercise exists to produce.

18. No timeouts, no connection-pool discipline on `db.*`; no metrics/tracing/logs

Unbounded queries and connection waits hang workers; with zero structured logs, duplicates/orphans/staleness are undiagnosable. Emit structured logs with `event_id` / `order_id`, counters for each outcome, and latency histograms.

P3 — Low: hygiene

19. `order.refunded += …` assumes attribute exists (schema drift crash) and `paid` is never validated.

20. No type hints, no tests ship with the handler, hard-coded strings instead of an enum; `db.save(order)` unconditionally rewrites the whole row (write amplification, blind overwrite of concurrent edits).

21. PII (email address) passed around without need; no redaction in logs.

Scoring summary

TIER	COUNT	ALL ARE RELEASE-BLOCKING?
P0 Critical	6	Yes — wrong money / forgeable
P1 High	7	Yes — corruption under real traffic
P2 Medium	5	Yes — operationally blind
P3 Low	3	No — fix in the rewrite

Corrected implementation

Design principles: **verify → dedupe → transact → recompute → enqueue → ack.** The handler never decides state incrementally; it appends the unique event and **recomputes** the order's state from its complete event log ordered by `sent_at`. That single decision makes out-of-order delivery, duplicate delivery, and replay all harmless, and keeps the code explainable in a five-minute interview.

```
import hmac, hashlib, json
from decimal import Decimal

PROCESSED = "processed_events"   # unique event_id — the idempotency key
ORDERS    = "orders"
EVENTS    = "webhook_events"     # append-only per order, indexed (order_id, sent_at)

def verify_signature(raw_body: bytes, signature_header: str | None) -> bool:
    if not signature_header:
        return False
    expected = hmac.new(settings.GATEWAY_WEBHOOK_SECRET.encode(), raw_body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature_header)

def parse_amount_paise(amount, api_version: str) -> int:
    if api_version == "v1":                      # rupees, decimal string
        return int((Decimal(str(amount)) * 100).to_integral_value())
    if api_version == "v2":                      # paise, integer
        return int(amount)
    raise ValueError(f"unsupported api_version: {api_version}")

@app.post("/webhook")
async def webhook(req: Request):
    raw = await req.body()
    if len(raw) > 256 * 1024:
        return JSONResponse({"error": "payload too large"}, status_code=413)
    if not verify_signature(raw, req.headers.get("X-Gateway-Signature")):
        return JSONResponse({"error": "invalid signature"}, status_code=401)
    try:
        e = json.loads(raw)
        amount_paise = parse_amount_paise(e["amount"], e["api_version"])
        event_type = e["type"]; assert event_type in VALID_TYPES
        event_id, order_id = e["event_id"], e["order_id"]
        sent_at = parse_ts(e["sent_at"])         # aware datetime, UTC
    except Exception:
        return JSONResponse({"error": "malformed event"}, status_code=400)

    async with db.transaction() as tx:
        # 1) idempotency: first delivery wins, retries are acknowledged and dropped
        inserted = await tx.execute(
            f"INSERT OR IGNORE INTO {PROCESSED}(event_id, order_id) VALUES (?,?)",
            (event_id, order_id))
        if inserted.rowcount == 0:
            return JSONResponse({"status": "duplicate", "event_id": event_id}, status_code=200)

        order = await tx.fetch_one(f"SELECT * FROM {ORDERS} WHERE order_id=? FOR UPDATE", (order_id,))
        if order is None:
            await tx.execute("INSERT INTO orphan_events(...) VALUES (...)")  # report, don't process
            return JSONResponse({"status": "orphan_recorded", "event_id": event_id}, status_code=200)

        # 2) append to the order's immutable event log
        await tx.execute(f"INSERT INTO {EVENTS}(event_id, order_id, type, amount_paise, sent_at) VALUES
(?,?,?,?,?)",
                         (event_id, order_id, event_type, amount_paise, sent_at))

        # 3) recompute terminal state from the FULL log in event-time order
        events = await tx.fetch_all(
            f"SELECT type, amount_paise, sent_at FROM {EVENTS} WHERE order_id=? ORDER BY sent_at,
event_id",
            (order_id,))
        new = recompute_order_state(order.amount_paise, events, cutoff_ist=SETTLEMENT_CUTOFF)
        #    recompute: failures stale after capture; authorization overridden by later
        #    capture; refunds move money only on refund.processed; >= for REFUNDED;
        #    EXPIRED if latest authorization older than cutoff-7d; mismatch flag if
        #    captured != order amount.
        await tx.execute(
            f"UPDATE {ORDERS} SET status=?, captured_paise=?, refunded_paise=?, mismatch=?,
```

```
    version=version+1 WHERE order_id=?",
                (new.state, new.captured_paise, new.refunded_paise, new.mismatch, order_id))
            await publish_outbox(tx, topic="order.status_changed",
                                 key=order_id, payload={"order_id": order_id, "status": new.state})
        # 4) after commit, workers send the customer email asynchronously (idempotent send)
        return JSONResponse({"status": "accepted", "event_id": event_id}, status_code=202)
```

Why this stands up in review: exactly-once effect via idempotent insert; correct under reordering via recompute; money in integer paise; orphans quarantined; signatures verified; email decoupled via the outbox; every outcome returns an honest status code so retries and dead-lettering behave. Residual trade-off: recompute is O(k) in events-per-order (k is single digits in practice); for pathological k, swap in a commutative fold — the seam is `recompute_order_state`, already isolated and unit-tested in `tests/test_engine.py` of this repository.